



Древняя магия каждый день

Пять принципов FP в пяти языках

Павел Аргентов

Evrone.ru

FP Guru



Павел Аргентов

- много опыта
- разрабатывал всё
- FP с 2002 г.

Evrone.ru: разработка
разработчиков

ФП: пять конкретных свойств кода

Смысл: описание результата, а не микроменеджмент процесса.

$$(\lambda x.M) N \rightarrow_{\beta} M[x := N]$$

```
[1,2,3].map(x ⇒ x * 2)
```

```
const add = x ⇒ y ⇒ x + y
```

```
const x = 1; x = 2 // TypeError
```

Пять языков

OCaml

язык богов (тм)

Scala

почти OCaml

Python

язык МЧС

JS

вездесущ

Rust

всемогуш

Декларативность

Декларативность

- Императивный код — микроменеджмент.
- Декларативный код — спецификация.

SELECT ... WHERE vs for ... in
for ... end vs list.map

Декларативность

- Людям — **понятность**
- Машинам — **законы оптимизации**

* для работы понадобится все остальные свойства **FP**

Декларативность: задача

Сумма скидок для заказов дороже 100, скидка 10%

Декларативность: OCaml

```
let total_discount orders =  
  orders  
  ▷ filter (fun o → o.price > 100.0)  
  ▷ map (fun o → o.price *. 0.1)  
  ▷ fold_left ( +. ) 0.0
```

Декларативность: Scala

```
def totalDiscount(orders: List[Order]): Double =  
  orders  
    .filter(_ .price > 100.0)  
    .map(_ .price * 0.1)  
    .sum
```

Декларативность: Rust

```
fn total_discount(orders: &[Order]) → f64 {  
    orders.iter()  
        .filter(|o| o.price > 100.0)  
        .map(|o| o.price * 0.1)  
        .sum()  
}
```

Декларативность: Python

```
def total_discount(orders):  
    return sum(o.price * 0.1 for o in orders if o.price > 100.0)
```

Декларативность: JS

```
const totalDiscount = (orders) =>  
  orders  
    .filter((o) => o.price > 100)  
    .map((o) => o.price * 0.1)  
    .reduce((acc, d) => acc + d, 0);
```

Выражения вместо инструкций

Выражения вместо инструкций

- Инструкции производят **эффект**
- Выражения вычисляются в **значения**
- Значения можно **компоновать**

Выражения вместо инструкций: задача

Классификация числа — вернуть метку без промежуточной переменной.

Выражения вместо инструкций: OCaml

```
let classify n =  
  match n with  
  | n when n < 0 → "negative"  
  | 0             → "zero"  
  | _             → "positive"
```

```
let label = classify (-5)  
  ▷ String.uppercase_ascii
```

Выражения вместо инструкций: Scala

```
def classify(n: Int): String =  
  n match {  
    case n if n < 0 ⇒ "negative"  
    case 0           ⇒ "zero"  
    case _           ⇒ "positive"  
  }  
  
val label = classify(-5).toUpperCase
```

Выражения вместо инструкций: Rust

```
fn classify(n: i32) → &'static str {  
    match n {  
        n if n < 0 ⇒ "negative",  
        0          ⇒ "zero",  
        _          ⇒ "positive",  
    }  
}  
  
let label = classify(-5).to_uppercase();
```

Выражения вместо инструкций: Python

```
def classify(n: int) → str:  
    return (  
        "negative" if n < 0 else  
        "zero"     if n == 0 else  
        "positive"  
    )  
label = classify(-5).upper()  
# match n: ... → match — инструкция
```

Выражения вместо инструкций: JS

```
const classify = (n) =>  
  n < 0 ? "negative"  
  : n === 0 ? "zero"  
  : "positive";
```

```
const label = classify(-5).toUpperCase();  
// if (...) {} → инструкция
```

Функции как значения

Функции как значения

- Функция — **first-class value**: передаётся аргументом, связывается с переменной, возвращается из функции
- HOF — принимает или возвращает функцию:
map, filter, fold
- Замыкание — захват лексического окружения
- Каррирование — **$f(a, b) \rightarrow f(a)(b)$**

Функции как значения: задача

Создать умножитель с коэффициентом и применить к списку.

Функции как значения: OCaml

(* каррирование по умолчанию *)

```
let multiply factor x = x * factor
```

```
let triple = multiply 3
```

(* замыкание *)

```
let make_adder base = fun x → base + x
```

```
let add10 = make_adder 10
```

(* HOF *)

```
let result = List.map triple [1; 2; 3; 4; 5]
```

Функции как значения: Scala

```
// каррирование (multiple param lists)
def multiply(factor: Int)(x: Int) = factor * x
val triple: Int ⇒ Int = multiply(3)

// замыкание
val base = 10
val addBase: Int ⇒ Int = x ⇒ x + base

// HOF
val result = List(1, 2, 3, 4, 5).map(triple)
```

Функции как значения: Rust

```
use auto_curry::curry;  
// #[curry] – процедурный макрос, каррирует функцию  
#[curry]  
fn multiply(factor: i32, x: i32) → i32 { factor * x }  
let triple: impl Fn(i32) → i32 = multiply(3);
```

Функции как значения: Rust

```
// triple – multiply, каррированный по factor
// add_base – замыкание: base захвачен из окружения
let base = 10;
let add_base: impl Fn(i32) → i32 = move |x| x + base;

// HOF: map принимает функцию аргументом
let result: Vec<i32> =
    vec![1, 2, 3, 4, 5].into_iter().map(triple).collect();
```

Функции как значения: Python

```
from functools import partial
def multiply(factor, x): return factor * x
triple = partial(multiply, 3) # каррирование
# HOF: map принимает функцию аргументом
result = list(map(triple, [1, 2, 3, 4, 5]))
# [3, 6, 9, 12, 15]
```

Функции как значения: Python

```
# result = list(map(triple, [1, 2, 3, 4, 5]))
# декоратор = HOF: fn → fn
def logged(fn):
    def wrapper(*a): return fn(*a)
    return wrapper
@logged
def triple2(x): return x * 3
```

Функции как значения: JS

```
import * as R from "ramda";  
// R.curry — каррирование через библиотеку Ramda  
const multiply = R.curry((factor, x)  $\Rightarrow$  factor * x);  
const triple = multiply(3);  
// addBase — замыкание: base захвачен из окружения  
const base = 10;  
const addBase = (x)  $\Rightarrow$  x + base;
```

Функции как значения: JS

```
// const triple = multiply(3)
// const addBase = (x) => x + base
// HOF: map принимает функцию аргументом
const result = [1, 2, 3, 4, 5].map(triple);
// [3, 6, 9, 12, 15]
```


Ссылочная прозрачность

Ссылочная прозрачность

- **Чистая функция:** возвращаемое значение зависит только от аргументов; **нет побочных эффектов.**
- Следствия: мемоизация; **независимость порядка вычислений**; data race-free **параллелизм.**
- **Эффекты** неизбежны (I/O, время, состояние).
Стратегия — **изоляция** на границах системы.

Ссылочная прозрачность: задача

Функция скидки — чистая и нечистая версии.

Ссылочная прозрачность: OCaml

```
let current_discount = ref 0.1
(* нечистая: глобальное состояние *)
let price_with_global_discount price =
  price *. (1.0 -. !current_discount)

(* чистая *)
let apply_discount rate price =
  price *. (1.0 -. rate)

let discounted = apply_discount 0.1 100.0
```

Ссылочная прозрачность: Scala

// чистая

```
def applyDiscount(rate: Double, price: Double): Double =  
    price * (1.0 - rate)
```

// эффекты — в типе

```
def withLog(rate: Double, price: Double): IO[Double] =  
    IO.println("...") *> IO.pure(applyDiscount(rate, price))
```

Ссылочная прозрачность: Rust

// чистая

```
fn apply_discount(rate: f64, price: f64) → f64 {  
    price * (1.0 - rate)  
}
```

// нечистая: &mut в сигнатуре

```
fn apply_discount_mut(rate: f64, price: &mut f64) {  
    *price *= 1.0 - rate;  
}
```

Ссылочная прозрачность: Python

```
current_discount = 0.1
```

```
# нечистая: зависит от глобального состояния
```

```
def price_with_global_discount(price: float) → float:  
    return price * (1 - current_discount)
```

```
# чистая
```

```
def apply_discount(rate: float, price: float) → float:  
    return price * (1 - rate)
```

Ссылочная прозрачность: JS

// нечистая: захватывает внешнее состояние

```
let taxRate = 0.2;
```

```
const priceWithTax = (price) ⇒ price * (1 + taxRate);
```

// чистая

```
const applyDiscount = (rate) ⇒ (price) ⇒ price * (1 - rate);
```


Иммутабельность

Иммутабельность

- `const` / `val` — запрет перезаписи переменной.
Иммутабельность — **запрет мутации самого значения**.
- Гарантия: **компилятором** (OCaml `let`, Rust `let`) / **соглашением** (Scala `val`) / **опционально** (Python `frozen=True`, JS `Object.freeze`).
- Persistent data structures: **structural sharing**.
Обновление поля — **$O(\log n)$** , не **$O(n)$** .

Иммутабельность

Обновить одно поле записи без изменения оригинала.

Иммутабельность: OCaml

(* record иммутабелен по умолчанию *)

```
type user = {  
  name: string;  
  age:  int;  
}
```

```
let user = { name = "Alice"; age = 30 }
```

```
let older = { user with age = 31 }
```

(* user не изменился *)

Иммутабельность: Scala

```
case class User(name: String, age: Int)
```

```
val user = User("Alice", 30)
```

```
val older = user.copy(age = 31)
```

```
// user.age = 31 → error: reassignment to val
```

Иммутабельность: Rust

```
struct User { name: String, age: u32 }
```

```
let user = User { name: "Alice".into(), age: 30 };
```

```
let older = User { age: 31, ..user };
```

```
let mut draft = User { name: "Bob".into(), age: 25 };  
draft.age = 26; // только let mut допускает мутацию
```

Иммутабельность: Python

```
from dataclasses import dataclass, replace
```

```
@dataclass(frozen=True)
```

```
class User:
```

```
    name: str
```

```
    age: int
```

```
user = User("Alice", 30)
```

```
older = replace(user, age=31)
```

```
user.age = 31 # FrozenInstanceError
```

Иммутабельность: JS

```
"use strict";
```

```
const user = Object.freeze({ name: "Alice", age: 30 });
```

```
const older = { ...user, age: 31 };
```

```
user.age = 31; // TypeError: Cannot assign to read only property
```


Итог: пять свойств × пять языков

	OCaml	Scala	Rust	Python	JS
Декларативность	+++	+++	+++	++	++
Выражения	+++	++	+++	+	+
Функции как значения	+++	+++	++	++	+++
Ссылочная прозрачность	+++	++	++	+	+
Иммутабельность	+++	++	+++	+	+

+++ — в дизайне языка, идиоматично ++ — поддерживается, требует дисциплины + — возможно, но нетипично

Задание на дом

1. **Какие** из пяти свойств присутствуют в вашем текущем проекте?
2. Какие из них язык гарантирует **by design**, а какие — только конвенцией?
3. Где явная иммутабельность или декларативный стиль снизили бы **когнитивную нагрузку** в последнем PR?



Павел Аргентов

Tg: @argent_smith

2026

